

can check out more at *netinet/in.h* for kernel *linux/in.h*:

- IPPROTO_IP = 0 Dummy protocol for TCP
- IPPROTO_ICMP = 1 Internet Control Message Protocol
- IPPROTO_IGMP = 2 Internet Group Management Protocol
- IPPROTO_TCP = 6 Transmission Control Protocol
- IPPROTO_UDP = 17 User Datagram Protocol
- IPPROTO_IPV6 = 41 IPv6-in-IPv4 tunnelling
- IPPROTO_SCTP = 132 Stream Control Transport Protocol
- IPPROTO_RAW = 255 Raw IP packets

Now, let's see Netfilter in action. Our tiny filter module will be going after the IP protocol 17 (UDP) packets. Let's name it *pfilter.c*. Since kernel modules run in kernel space and can crash your system, always be careful when writing such code.

We begin by making the necessary includes for a kernel module:

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/in.h>    /* For protocol names seen above */
#include <linux/ip.h>    /* Just using the IPv4 */
#include <linux/udp.h>
#include <linux/netfilter.h>
#include <linux/netfilter_ipv4.h>
```

These two hook-operation structures will be used to specify the hook-registration options for installing our callback function.

```
static struct nf_hook_ops netfilter_ops_in; /* NF_IP_PRE_ROUTING */
static struct nf_hook_ops netfilter_ops_out; /* NF_IP_POST_ROUTING */

struct sk_buff *s_buffer; /* We will access socket buffer through this pointer */
struct iphdr *ip_header; /* IP header will be accessed through this pointer */
struct udphdr *udp_header; /* Structure pointer to access UDP header */
```

Our single callback function is defined as follows. (We could have created multiple hook functions and registered them, but for our simple requirements, this is sufficient.) The function prototype is in *linux/netfilter.h*:

```
unsigned int main_hook(unsigned int hooknum, struct sk_buff *skb,
const struct net_device *in, const struct net_device *out,
int (*okfn)(struct sk_buff *))
{
    s_buffer = skb; /* Got the buffer */

    if(!s_buffer) { return NF_ACCEPT; }
```

```
    ip_header= ip_hdr (s_buffer);

    if (ip_header->protocol == IPPROTO_UDP) {
        udp_header = udp_hdr (s_buffer); /* We are not processing the UDP
header data here in situations we can use it */
        printk(KERN_INFO "Got UDP packet!\n");
        return NF_DROP;
    }
    return NF_ACCEPT;
}
```

Let's see what we have done in the above code. We copy the data to *s_buffer*, so we can process it as per our need. If the buffer is empty, return *NF_ACCEPT* to continue the traversal. Next, we check for the protocol in the IP header; if it is UDP, we copy the header from the buffer (in case we had wanted to do something more with it), and drop the packet by returning *NF_DROP*. We also print a kernel message, so we know the filter is working by monitoring the kernel logs. For all other packets, we're returning *NF_ACCEPT* to let Netfilter continue the processing.

Next, we need to make our kernel module register the function with the chosen hooks. Whenever a kernel module is loaded (during boot, or manually), it starts with a call to *init_module()*. Here, we use this initialisation function to create two hooks, one for incoming, and the other for outgoing packets. We then register the hooks (both our hooks register the same callback function, *main_hook()*). Our *cleanup_module()* function just un-registers the hooks. Let's look at the code:

```
int init_module()
{
    netfilter_ops_in.hook = main_hook;
    netfilter_ops_in.pf = PF_INET;
    netfilter_ops_in.hooknum = NF_INET_PRE_ROUTING;
    netfilter_ops_in.priority = NF_IP_PRI_FIRST;

    netfilter_ops_out.hook = main_hook;
    netfilter_ops_out.pf = PF_INET;
    netfilter_ops_out.hooknum =NF_INET_POST_ROUTING;
    netfilter_ops_out.priority = NF_IP_PRI_FIRST;

    nf_register_hook(&netfilter_ops_in); /* register NF_IP_PRE_ROUTING
hook */
    nf_register_hook(&netfilter_ops_out); /* register NF_IP_POST_ROUTING
hook */

    printk(KERN_INFO "MODULE INSERTED!\n"); /* Kernel just prints the
message to kernel messages */
    return 0;
}

void cleanup_module()
{

```